

RV32I 5-Stage Pipelined CPU 최종 구현 보고서

RISC-V A* Accelerator SoC 프로젝트 - CPU Core 최종 정리본

작성일: 2026-07-01 | 구현 언어: Verilog HDL | 검증 환경: Questa Altera Starter FPGA Edition 2025.2

항목	내용
프로젝트명	RV32I A* Accelerator SoC
CPU 목표	RV32I subset 기반 5-stage pipelined CPU 구현 및 검증
참고 교재	David A. Patterson, John L. Hennessy, Computer Organization and Design RISC-V Edition

1. 구현 목표와 현재 범위

본 보고서는 RV32I A* Accelerator SoC 프로젝트에서 구현한 RISC-V CPU core 의 최종 정리본이다. 초기에는 단일 사이클 형태의 instruction fetch, register file, ALU 동작을 확인하는 단계에서 출발했고, 이후 5-stage pipeline 구조, data hazard 처리, branch 처리, load/store, LUI, JAL, JALR 까지 확장했다.

현재 구현은 RV32I 전체 명령어 집합을 모두 포함하는 상용 CPU 가 아니라, 프로젝트 진행에 필요한 RV32I subset 을 직접 구현한 CPU core 이다. 다만 pipeline register, forwarding, stall, flush, writeback mux, instruction memory, data memory 를 독립 모듈로 구성하여 이후 MMIO bus 와 A* accelerator 를 연결할 수 있는 형태로 정리했다.

이번 최종 버전에서 새로 보강된 핵심은 LUI 와 jump 계열이다. LUI 는 큰 상수와 MMIO base address 를 만들기 위한 기반이고, JAL/JALR 은 loop, 함수 호출, 복귀 흐름을 다루기 위한 기본 PC 제어 명령어이다. 따라서 단순 ALU 명령어 검증에서 끝나지 않고, 프로그램 흐름을 바꾸는 control-flow 명령어까지 포함해 최종 통합 testbench 로 확인했다.

2. 최종 명령어 subset

표 1 은 현재 RTL 에서 decode 및 실행 경로를 갖는 명령어들을 정리한 것이다. opcode 는 instruction[6:0], funct3 는 instruction[14:12], funct7 은 instruction[31:25]에 해당한다. R-type 과 I-type ALU 명령어는 opcode 만으로 구분되지 않으므로 funct3/funct7 조합을 이용해 ALU control signal 을 만든다.

분류	명령어	opcode	funct3	funct7/특이사항	동작 요약
R	ADD	0110011	000	0000000	$rd = rs1 + rs2$
R	SUB	0110011	000	0100000	$rd = rs1 - rs2$
R	AND	0110011	111	0000000	$rd = rs1 \& rs2$
R	OR	0110011	110	0000000	$rd = rs1 rs2$
R	XOR	0110011	100	0000000	$rd = rs1 \wedge rs2$
R	SLL	0110011	001	0000000	$rd = rs1 \ll rs2$
I-ALU	ADDI	0010011	000	imm[11:0]	$rd = rs1 + imm$
I-ALU	SLLI	0010011	001	imm[11:5]=0000000	$rd = rs1 \ll shamt$
I-ALU	XORI	0010011	100	imm[11:0]	$rd = rs1 \wedge imm$
I-ALU	ORI	0010011	110	imm[11:0]	$rd = rs1 imm$
I-ALU	ANDI	0010011	111	imm[11:0]	$rd = rs1 \& imm$
U	LUI	0110111	-	instr[31:12] << 12	$rd = \text{upper immediate}$
Memory	LW	0000011	010	I-type imm	$rd = \text{DMEM}[rs1 + imm]$
Memory	SW	0100011	010	S-type imm	$\text{DMEM}[rs1 + imm] = rs2$
Branch	BEQ	1100011	000	B-type imm	if $rs1 == rs2$ then $PC = PC + imm$
Branch	BNE	1100011	001	B-type imm	if $rs1 \neq rs2$ then $PC = PC + imm$
Branch	BLT	1100011	100	B-type imm	signed $rs1 < \text{signed } rs2$
Branch	BGE	1100011	101	B-type imm	signed $rs1 \geq \text{signed } rs2$

Jump	JAL	1101111	-	J-type imm	rd = PC+4, PC = PC + imm
Jump	JALR	1100111	000	I-type imm	rd = PC+4, PC = (rs1 + imm) & ~1

3. Immediate 생성 방식

imm_gen 모듈은 instruction format 별 immediate 를 32-bit 값으로 sign-extension 또는 zero-positioned 형태로 만든다. PC-relative 명령어인 branch 와 JAL 은 immediate 의 최하위 bit 가 항상 0 이므로 decode 단계에서 1'b0 을 붙여 byte offset 형태로 만든다.

Format	사용 명령어	생성식	비고
I	LW, ADDI, ANDI, ORI, XORI, JALR	{{20{IR[31]}}, IR[31:20]}	12-bit signed immediate
S	SW	{{20{IR[31]}}, IR[31:25], IR[11:7]}	store offset
B	BEQ/BNE/BLT/BGE	{{20{IR[31]}}, IR[7], IR[30:25], IR[11:8], 1'b0}	branch target offset
U	LUI	{IR[31:12], 12'b0}	upper immediate
J	JAL	{{12{IR[31]}}, IR[19:12], IR[20], IR[30:21], 1'b0}	jump target offset

4. 전체 pipeline 구조

CPU 는 IF, ID, EX, MEM, WB 의 5 단계 pipeline 으로 구성했다. 각 stage 사이에는 IF/ID, ID/EX, EX/MEM, MEM/WB pipeline register 를 두어 다음 cycle 로 필요한 control signal 과 data signal 을 전달한다.

```

IF : PC -> IMEM -> instruction fetch
ID : decode, register read, immediate generation, branch/jump target decision
EX : ALU operation, EX forwarding, store data forwarding
MEM : data memory read/write
WB : ALU/load/PC+4/imm 중 하나를 register file 에 writeback

```

이번 구조의 가장 중요한 설계 결정은 branch 와 jump target 을 ID stage 에서 결정한다는 점이다. 이렇게 하면 branch/jump 가 결정된 뒤 잘못 fetch 된 instruction 을 IF/ID 에서 NOP 으로 flush 하여

control hazard penalty 를 줄일 수 있다. 다만 ID stage 에서 rs1/rs2 값을 바로 사용하기 때문에 branch 와 JALR 에는 ID-stage forwarding 및 stall 제어가 추가로 필요하다.

5. Control signal 설계

Control 모듈은 opcode 를 기준으로 major control signal 을 생성한다. 초기 구조에서는 branch signal 하나로 branch 와 jump 를 같이 처리하려 했지만, 최종 구조에서는 branch, jal, jalr 을 분리했다. 이 분리는 매우 중요하다. branch 는 조건 비교가 필요하고 rd write 가 없지만, JAL/JALR 은 조건 비교 없이 PC 를 바꾸고 rd 에 PC+4 를 저장해야 하기 때문이다.

명령어 그룹	branch	jal	jalr	alusrc	aluop	memread	memwrite	regwrite	result_select
R-type	0	0	0	0	10	0	0	1	00 ALU
I-type ALU	0	0	0	1	11	0	0	1	00 ALU
LW	0	0	0	1	00	1	0	1	01 load
SW	0	0	0	1	00	0	1	0	don't care
Branch	1	0	0	0	01	0	0	0	don't care
LUI	0	0	0	1	00	0	0	1	11 imm
JAL	0	1	0	0	00	0	0	1	10 PC+4
JALR	0	0	1	1	00	0	0	1	10 PC+4

result_select 는 최종 WB stage 에서 register file 에 쓸 값을 고르는 2-bit signal 이다. 기존 memtoreg 만으로는 ALU result 와 load data 만 구분할 수 있었기 때문에 LUI 와 JAL/JALR 을 자연스럽게 구현하기 어렵다. 최종 구조에서는 00=ALU result, 01=load data, 10=PC+4, 11=immediate 로 확장했다.

6. 모듈별 구현 정리

모듈	역할	구현 내용
cpu_core.v	전체 top module	각 stage module 을 연결하고 PC, hazard, forwarding, WB mux 를 통합한다.
PC.v	next PC 선택	stall 이 없으면 pc_redirect 에 따라 target_address 또는 PC+4 를 선택한다.
imem.v	instruction memory	\$readmemh 로 hex 파일을 읽고 PC[11:2]를 word index 로 사용한다.
IF_ID_reg.v	IF/ID register	stall 이면 hold, flush 이면 NOP(32'h00000013)을 삽입한다.
Control.v	main control	opcode 기반으로 branch/jal/jalr, memory, writeback control 을 생성한다.
reg_file.v	32 개 register	posedge write, combinational read, WB bypass 를 포함한다.
imm_gen.v	immediate generator	I/S/B/U/J format immediate 를 32-bit 로 확장한다.
ID_fowarding_unit.v	ID-stage hazard/forwarding	branch 와 JALR 이 ID 에서 최신 rs 값을 사용하도록 stall/forwarding select 를 만든다.
branch_detect.v	branch/jump redirect	branch condition, pc_redirect, IF/ID flush 를 결정한다.
bta_jta.v	target address 계산	branch/JAL 은 D_PC+imm, JALR 은 (rs1+imm)&~1 을 사용한다.
ID_EX_reg.v	ID/EX register	EX/MEM/WB 에 필요한 control 과 data 를 전달하며 hazard bubble 시 control 을 0 으로 만든다.
alu_controller.v	ALU control	aluop 와 funct 조합으로 실제 ALU operation code 를 만든다.
alu.v	ALU + EX forwarding mux	ADD/SUB/AND/OR/XOR/SLL 및 store data forwarding 을 처리한다.
EX_forwarding_unit.v	EX-stage forwarding	EX/MEM, MEM/WB 의 rd 와 ID/EX rs 를 비교해 ALU 입력 mux 를 제어한다.

EX_MEM_reg.v	EX/MEM register	ALU result, store data, rd, result_select, imm, PC 를 MEM stage 로 전달한다.
dmem.v	data memory	word-addressed memory 이며 i_alu_out[ADDR_WIDTH+1:2]를 word index 로 사용한다.
MEM_WB_reg.v	MEM/WB register	WB mux 에 필요한 ALU/load/PC/imm/result_select 를 저장한다.

7. Hazard 처리

7.1 Load-use data hazard

load-use hazard 는 ID/EX stage 의 instruction 이 LW 이고, 다음 instruction 이 같은 rd 를 rs1 또는 rs2 로 사용할 때 발생한다. load data 는 MEM stage 에서 나오므로 바로 다음 EX stage 에서 사용할 수 없다. 따라서 hazard_detect 모듈은 PC 와 IF/ID register 를 hold 하고, ID/EX 에는 bubble 을 삽입한다.

```
if (E_memread && E_rd != x0 &&
    ((D_use_rs1 && D_RS1 == E_rd) || (D_use_rs2 && D_RS2 == E_rd))) begin
    PCwrite_load = 0;
    IF_ID_write_load = 0;
    load_hazard_bubble = 1;
end
```

7.2 EX-stage forwarding

일반 ALU data hazard 는 EX_forwarding_unit 에서 처리한다. EX/MEM 의 rd 또는 MEM/WB 의 rd 가 현재 EX stage 의 rs1/rs2 와 같으면 ALU 입력 mux 가 register file 값 대신 최신 결과를 선택한다. 최종 구조에서는 M_forward_data 를 사용해 ALU result 뿐 아니라 LUI immediate 결과와 JAL/JALR 의 PC+4 결과도 다음 instruction 으로 forwarding 될 수 있게 했다.

7.3 ID-stage branch/JALR hazard

branch 와 JALR 은 ID stage 에서 target 또는 condition 을 결정한다. 따라서 EX stage 보다 더 이른 시점에 최신 rs 값을 필요로 한다. EX stage 에 있는 바로 앞 instruction 의 결과는 아직 사용할 수 없으므로 stall 이 필요하고, MEM stage 에 있는 instruction 의 결과는 M_forward_data 또는 load_data 를 통해 ID 로 전달한다.

상황	예시	처리
branch 가 EX stage 결과를 바로 필요로 함	add x5,... 다음 beq x5,x0,label	PC/IF_ID hold, ID/EX bubble
branch 가 MEM stage ALU/LUI/JAL 결과를 필요로 함	addi x5,... 이후 beq x5,...	ID forwarding 으로 M_forward_data 선택
branch 가 MEM stage load 결과를 필요로 함	lw x5,... 이후 beq x5,...	추가 stall 후 load_data 선택
JALR 이 rs1 최신 값을 필요로 함	addi x10,... 다음 jalr x6,0(x10)	branch 와 동일한 ID-stage rs1 hazard 처리

8. Branch, JAL, JALR 의 PC 제어

최종 PC 제어는 pc_redirect 와 target_address 로 통합했다. branch 는 조건이 참일 때만 redirect 되고, JAL/JALR 은 조건 비교 없이 redirect 된다. redirect 가 발생하면 IF/ID register 에는 NOP 을 넣어 잘못 fetch 된 instruction 을 제거한다.

```
pc_redirect = ((branch && branch_condition) || jal || jalr) && !ID_EX_stall;

target_address =
    jalr ? ((D_forwarding1 + D_imm) & 32'hffff_fffe)
    : (D_PC + D_imm);

next_PC =
    (PCwrite_load && PCwrite_branch) ?
    (pc_redirect ? target_address : PC + 4) :
    PC;
```

JAL/JALR 은 PC 를 바꾸는 동시에 rd 에 PC+4 를 저장한다. 이때 PC 변경은 ID stage 에서 바로 결정되며, rd 저장은 pipeline 을 따라 WB stage 에서 수행된다. 즉 jump 를 WB 까지 기다리는 구조가 아니라, control-flow 변경과 return address 저장이 서로 다른 stage 에서 병렬적으로 처리되는 구조이다.

9. LUI 와 writeback mux

LUI 는 ALU 결과를 사용할 필요가 없다. imm_gen 에서 {IR[31:12], 12'b0} 형태로 32-bit immediate 를 만들고, result_select=2'b11 을 통해 WB stage 에서 해당 immediate 를 rd 에 기록한다. JAL/JALR 은 result_select=2'b10 을 사용해 WB stage 에서 PC+4 를 rd 에 기록한다. 이 확장으로 기존의 memtoreg 1-bit mux 구조보다 writeback 경로가 명확해졌다.

result_select	WB 데이터	사용 명령어
00	WB_alu_out	R-type, I-type ALU
01	WB_load_data	LW
10	WB_PC + 4	JAL, JALR
11	WB_imm	LUI

10. 최종 통합 testbench

최종 보고서용 검증은 tb_final_pipeline_proof.v 와 program_final_pipeline_proof.hex 로 수행했다. 이 테스트는 단순히 하나의 명령어만 확인하는 것이 아니라, 여러 hazard 와 control-flow 가 섞인 프로그램을 실행한 뒤 register file 과 data memory 의 최종 상태를 검사한다.

검증 항목	테스트 방식	확인 포인트
LUI	lui x1, 0x12345	x1 = 0x12345000
LUI 직후 사용	addi x2, x1, 0x678	x2 = 0x12345678, forwarding 확인
JAL	jal x3, func	x3 = return address, wrong-path flush
JALR return	jalr x0, 0(x3)	함수에서 원래 흐름으로 복귀
JALR register jump	addi x10, 48 이후 jalr x6, 0(x10)	x6 = PC+4, target jump
Load-use hazard	lw x14 후 add x15, x14, x4	stall 후 x15 = 118
Store/load memory	sw x15, 12(x0), sw x2, 16(x0), lw x26, 16(x0)	dmem[3], dmem[4] 확인

Branch taken/flush	BEQ/BNE/BLT/BGE	wrong-path register 가 0 유지
ALU R/I-type	SUB/AND/OR/XOR/SLLI/ANDI/ORI/XORI	x20, x22, x24, x25, x28~x31 확인

11. 최종 검증 결과

최종 통합 testbench 는 모든 register/memory checkpoint 를 OK 로 통과했고, 최종 PASS 메시지를 출력했다. readmem 관련 warning 은 cpu_core 내부 IMEM 기본 parameter 가 program.hex 를 먼저 찾기 때문에 발생하지만, testbench 가 곧바로 program_final_pipeline_proof.hex 로 IMEM 을 덮어쓰기 때문에 검증 결과에는 영향을 주지 않는다.

```

VSIM 3> run -all
# OK: x0 = 0 (0x00000000)
# OK: x1 = 305418240 (0x12345000)
# OK: x2 = 305419896 (0x12345678)
# OK: x3 = 12 (0x0000000c)
# OK: x4 = 111 (0x0000006f)
# OK: x5 = 7 (0x00000007)
# OK: x6 = 24 (0x00000018)
# OK: x7 = 0 (0x00000000)
# OK: x8 = 0 (0x00000000)
# OK: x9 = 26 (0x0000001a)
# OK: x10 = 48 (0x00000030)
# OK: x11 = 0 (0x00000000)
# OK: x12 = 0 (0x00000000)
# OK: x13 = 305419903 (0x1234567f)
# OK: x14 = 7 (0x00000007)
# OK: x15 = 118 (0x00000076)
# OK: x16 = 118 (0x00000076)
# OK: x17 = 0 (0x00000000)
# OK: x18 = 3 (0x00000003)
# OK: x19 = 3 (0x00000003)
# OK: x20 = 4 (0x00000004)
# OK: x21 = 1 (0x00000001)
# OK: x22 = 3 (0x00000003)
# OK: x23 = 4 (0x00000004)
# OK: x24 = 0 (0x00000000)
# OK: x25 = 9 (0x00000009)
# OK: x26 = 305419896 (0x12345678)
# OK: x27 = 305419897 (0x12345679)
# OK: x28 = 5 (0x00000005)
# OK: x29 = 3 (0x00000003)
# OK: x30 = 0 (0x00000000)
# OK: x31 = 5 (0x00000005)
# OK: dmem[0] = 7 (0x00000007)
# OK: dmem[1] = 3 (0x00000003)
# OK: dmem[2] = 9 (0x00000009)
# OK: dmem[3] = 118 (0x00000076)
# OK: dmem[4] = 305419896 (0x12345678)
# PASS: final integrated pipeline proof - ALU, LUI, jumps, branches, load-use, forwarding, LW/SW
# ** Note: $finish      : D:/Programs/vscode_workspace/Soc_Project/Risc_V/code_6_27-7_01/tb_final_pipeline_proof.v(105)
#   Time: 1815 ns  Iteration: 1  Instance: /tb final pipeline proof

```

그림 1. 최종 통합 testbench PASS 로그

분류	대표 확인값
LUI/ADDI	x1=0x12345000, x2=0x12345678
JAL/JALR	x3=12, x6=24, x7/x8/x11/x12=0
Load-use	x14=7, x15=118
Branch	x17=0, x20=4, x22=3, x24=0, x25=9
Memory	dmem[3]=118, dmem[4]=0x12345678
ALU	x28=5, x29=3, x30=0, x31=5

12. Waveform 캡처 위치

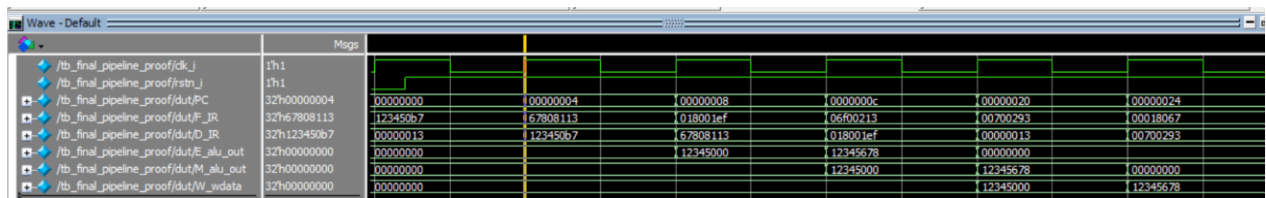


그림 2. Waveform 캡처: 기본 파이프라인 신호

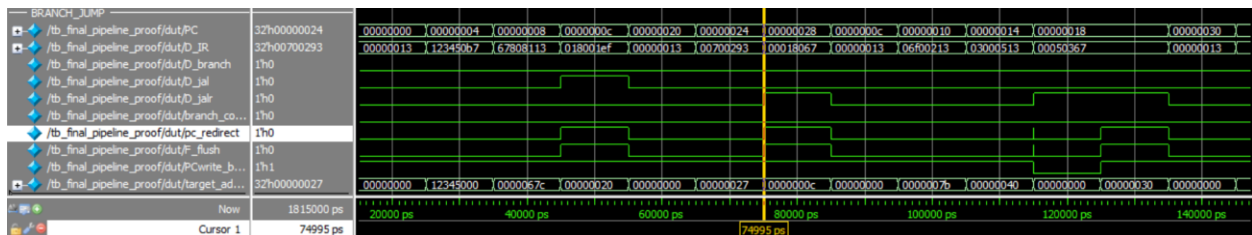


그림 3. Waveform 캡처: target address 로 브랜치 및 점프

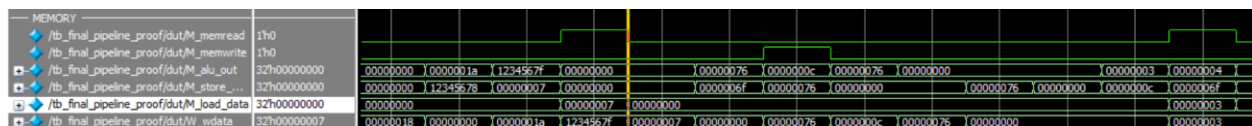


그림 4. Waveform 캡처: load/store

13. 최종 구현의 의미

이번 구현은 단순히 instruction 하나가 동작하는지를 확인한 수준을 넘어서, pipeline CPU 에서 실제로 문제가 되는 data hazard 와 control hazard 를 직접 다루었다는 점에 의미가 있다. 특히 branch 와 JALR 을 ID stage 에서 처리하면서 생기는 추가 hazard 를 별도 forwarding/stall logic 으로 해결했고, Lui/Jal/Jalr 을 위해 writeback mux 를 확장했다.

A* accelerator SoC 관점에서는 아직 MMIO bus 와 accelerator datapath 가 통합되지는 않았지만, CPU 가 큰 상수 생성(Lui), memory access(Lw/Sw), branch loop, jump/return 흐름을 다룰 수 있게 되었기 때문에 이후 firmware 형태의 프로그램을 올릴 기반이 마련되었다.

14. 결론

최종 RTL 은 5-stage pipelined RV32I subset CPU 로서 ALU 연산, load/store, branch, Lui, Jal, Jalr, data forwarding, load-use stall, branch/jump flush 를 포함한다. 최종 통합 testbench 는 ALU, Lui, jumps, branches, load-use, forwarding, Lw/Sw 를 모두 포함한 프로그램을 실행했고, register file 및 data memory checkpoint 가 모두 기대값과 일치했다.

따라서 현재 단계의 CPU core 는 A* accelerator SoC 통합을 위한 기본 CPU 실행 기반으로 사용할 수 있는 상태까지 도달했다고 정리할 수 있다.